

De los pasos que me dijo para implement basic authentication, estos son los hechos y los
no hechos por mí hasta ahora: 26

Implementación de autenticación básica

I have an [ASP.NET Core Web API backend](#) project running in a Docker container plus a sql server running in another docker container. It's all for a booking app. I've already successfully implemented CRUD endpoints in Controllers writing the corresponding code using Visual Studio IDE. And I've also successfully tested all "GET all" and POST endpoints using both Swagger and Postman.

The structure of my project is the following:

reservas-app folder is the root of it all and contains four folders: 'backend', 'deployment', 'frontend' and 'cronservice'.

'cronservice' folder currently only contains .git folder, .gitignore file and README.md file.

'frontend' folder contains all the basics for an Angular frontend, but I have not implemented anything that the user can see or click on, I probably have just the typical Angular frontend template. To be more specific, my 'frontend' folder currently contains .git folder, .vscode folder, node_modules folder, public folder, src folder, .editorconfig file, .gitignore file, angular.json file, Dockerfile, package.json file, package-lock.json file, README.md file, tsconfig.app.json file, tsconfig.json file and tsconfig.spec.json file.

To the contrary, I've already done a lot of coding in my backend project, for instance the code for implementing the CRUD endpoints. What I have inside this backend folder is currently the following:

.git folder, .github folder, .vs folder, .vscode folder, ReservasApi folder, ReservasApi.Tests folder, .dockerignore file, .gitignore file, backend.sln file and Docker file.

Inside ReservasApi folder I have the following:

bin folder, Controller folder (CRUD endpoints files and code), Data folder, Eventos folder (for Kafka events, but all Kafka code is currently commented so it's like it does not exist), Migrations folder, Modelos folder (models for tables of bd), obj folder, Properties folder, appsettings.Development.json file, appsettings.json file, Program.cs file, ReservasApi.csproj file, ReservasApi.csproj.user file and ReservasApi.http file.

Also worth mentioning my backend code is c sharp i.e. c# code.

Now I've been told to "implement basic authentication", I guess they are referring to user register plus user login. Please **guide** me through the process of "**implementing basic authentication**".


```
ReservasApi.csproj X
C: > Users > iago.cadavidgonzalez > Proyectos > reservas-app > backend > P
1  <Project Sdk="Microsoft.NET.Sdk.Web">
2
3      <PropertyGroup>
4          <TargetFramework>net9.0</TargetFramework>
5          <Nullable>enable</Nullable>
6          <ImplicitUsings>enable</ImplicitUsings>
7      </PropertyGroup>
```

Regarding "Your backend is .NET 6–8 (the process is the same)." in ReservasApi.csproj what I have is
<TargetFramework>net9.0</TargetFramework>

Great — .NET 9.0 works perfectly fine.

All instructions I gave are fully compatible with .NET 9, and there is nothing you need to change in the authentication steps because:

✓ ASP.NET Core identity & JWT authentication work the same in .NET 6, 7, 8, and 9

The differences between versions do *not* affect:

- JWT authentication
- Middleware pipeline (`UseAuthentication` , `UseAuthorization`)
- Controller attributes (`[Authorize]` , `[AllowAnonymous]`)
- Dependency injection
- Entity Framework Core authentication logic



So you can follow the implementation exactly as described.

Just verify one thing in Program.cs (if you're using minimal hosting)

Since .NET 6 Microsoft moved to the *minimal hosting model* and .NET 9 uses it by default.

Your Program.cs likely looks like:

```
csharp

var builder = WebApplication.CreateBuilder(args);

// Add services here

var app = builder.Build();

// Configure middleware here

app.Run();
```



En

efecto mi Program.cs tiene esas 3 líneas y entre las primeras dos tengo lo de add services:

```
5  var builder = WebApplication.CreateBuilder(args); // esto ya viene por defecto.
6
7  // 1. Add services to the container.
8
9  // Add controllers (this enables the [ApiController] controllers)
10 builder.Services.AddControllers(); // .AddOpenApi();
11 /*
12  .AddJsonOptions(opciones => // esto lo añado con la intención de que get api reservas
13  {
14      opciones.JsonSerializerOptions.ReferenceHandler = System.Text.Json.Serialization.ReferenceHandler.IgnoreCycles;
15      opciones.JsonSerializerOptions.WriteIndented = true;
16  });
17  */
18
19 // Add EF Core and configure SQL Server using the connection string from appsettings.json
20 builder.Services.AddDbContext<AppDbContext>(opciones =>
21     opciones.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection"))
22 );
23
24 // Add Swagger/OpenAPI support for testing your endpoints easily
25 builder.Services.AddEndpointsApiExplorer();
26 builder.Services.AddSwaggerGen(); // requires installation of the Swagger (Swashbuckle) NuGet package
27
28 // Register the Kafka producer in DI, para ello añado esta línea antes de var app = builder.Build();
29 //builder.Services.AddSingleton<IServicioProductorKafka, ProductorKafka>(); // ProductorKafka
30 // COMENTO LA LÍNEA ANTERIOR DE CARA A EMPEZAR CON TESTS UNITARIOS IGNORAR TODO LO DE KAFKA
31
32 var app = builder.Build(); // ya viene por defecto.
```


Entre la 2nda y 3era líneas tengo esto:

```
32 var app = builder.Build(); // ya viene por defecto.
33
34 // 2. Configure the HTTP request pipeline.
35 if (app.Environment.IsDevelopment()) // esto ya viene por defecto
36 {
37     //app.MapOpenApi(); este es el único que viene por defecto.
38     // Enable Swagger UI only in development :
39     app.UseSwagger(); // esto lo puse yo
40     app.UseSwaggerUI(); // esto lo puse yo
41 }
42
43 app.UseHttpsRedirection(); // ya viene por defecto.
44
45 app.UseAuthorization(); // lo puse yo.
46
47 // Map controller routes (this enables your /api/... endpoints)
48 app.MapControllers(); // lo puse yo.
49
50 app.Run(); // lo puse yo.
```

You only need to make sure:

```
csharp

// BEFORE app.Build()
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(/* ... */);

builder.Services.AddAuthorization();
builder.Services.AddScoped<AuthService>();

// AFTER app.UseRouting()
app.UseAuthentication();
app.UseAuthorization();
```

Página siguiente

In my backend Program.cs I don't have the following :

```
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
```

```
.AddJwtBearer(/* ... */);
```

```
builder.Services.AddAuthorization();
```

```
builder.Services.AddScoped<AuthService>();
```

Should I add it before `var app = builder.Build()` ?

Yes — **exactly**.

In ASP.NET Core (including .NET 9), all **service registrations** must be added **before**:

csharp

 Copy code

```
var app = builder.Build();
```

So you should insert the authentication setup **above** `app.Build()`, along with your other `builder.Services.*` lines.

NEXT PAGE

When I type in Program.cs (using Visual Studio Code) `builder.Services.AddAuth` it doesn't autocomplete the rest, which for instance does happen with `builder.Services.AddSwaggerGen()`

Ah — that's a common VS Code IntelliSense issue and also sometimes a missing using directive. Unlike `AddSwaggerGen()`, `AddAuthentication()` is an extension method that lives in `Microsoft.AspNetCore.Authentication` namespaces. If the compiler/IntelliSense doesn't know about it, you won't get autocomplete.

Here's how to fix it:

1 Add the proper `using` statements at the top of Program.cs

csharp

 Copy code

```
using Microsoft.AspNetCore.Authentication.JwtBearer;  
using Microsoft.IdentityModel.Tokens;  
using System.Text;
```

- `Microsoft.AspNetCore.Authentication.JwtBearer` → contains `AddAuthentication()` and `AddJwtBearer()`.
- `Microsoft.IdentityModel.Tokens` → needed for `SymmetricSecurityKey` and `TokenValidationParameters`.
- `System.Text` → needed for `Encoding.UTF8.GetBytes(...)`.

2 Make sure the NuGet package is installed

You need `Microsoft.AspNetCore.Authentication.JwtBearer`. In your backend folder run:

bash

 Copy code

```
dotnet add package Microsoft.AspNetCore.Authentication.JwtBearer
```

After adding it, restart VS Code so IntelliSense refreshes.

3 Check IntelliSense again

Now typing:

csharp

 Copy code

```
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
```


En mi ReservasApi.csproj no hay ningún PackageReference con Microsoft.AspNetCore.Authentication.JwtBearer, así que desde ReservasApi folder (padre del csproj file) abro cmd y ejecuto:

dotnet add package Microsoft.AspNetCore.Authentication.JwtBearer para añadir dicho PackageReference al csproj file.

```
C:\Windows\System32\cmd.exe
Microsoft Windows [Versión 10.0.26280.7171]
(c) Microsoft Corporation. Todos los derechos reservados.

C:\Users\Iago.cadavidgonzalez\Proyectos\reservas-app\backend\ReservasApi>dotnet add package Microsoft.AspNetCore.Authentication.JwtBearer

Compilación realizada correctamente en 3,9s
info : La validación de la cadena de certificados X.509 utilizará el almacén de confianza predeterminado seleccionado por .NET para la firma de código.
info : La validación de la cadena de certificados X.509 utilizará el almacén de confianza predeterminado seleccionado por .NET para la marca de tiempo.
info : Agregando PackageReference para el paquete "Microsoft.AspNetCore.Authentication.JwtBearer" al proyecto "C:\Users\Iago.cadavidgonzalez\Proyectos\reservas-app\backend\ReservasApi\ReservasApi.csproj".
info : GET https://api.nuget.org/v3/registrations-gr-sewver2/microsoft.aspnetcore.authentication.jwtbearer/index.json 348 ms
info : GET https://api.nuget.org/v3/registrations-gr-sewver2/microsoft.aspnetcore.authentication.jwtbearer/page/0.0.1-alpha/3.1.17.json 163 ms
info : GET https://api.nuget.org/v3/registrations-gr-sewver2/microsoft.aspnetcore.authentication.jwtbearer/page/3.1.18/6.0.11.json 468 ms
info : GET https://api.nuget.org/v3/registrations-gr-sewver2/microsoft.aspnetcore.authentication.jwtbearer/page/6.0.12/6.0.2.json 179 ms
info : GET https://api.nuget.org/v3/registrations-gr-sewver2/microsoft.aspnetcore.authentication.jwtbearer/page/8.0.3/10.0.0.json 151 ms
info : Restaurando paquetes para C:\Users\Iago.cadavidgonzalez\Proyectos\reservas-app\backend\ReservasApi\ReservasApi.csproj...
info : GET https://api.nuget.org/v2-flatcontainer/microsoft.aspnetcore.authentication.jwtbearer/index.json 147 ms
info : GET https://api.nuget.org/v3-flatcontainer/microsoft.aspnetcore.authentication.jwtbearer/10.0.0/microsoft.aspnetcore.authentication.jwtbearer.10.0.0.mupkg 188 ms
info : OK https://api.nuget.org/v3-flatcontainer/microsoft.aspnetcore.authentication.jwtbearer/10.0.0/microsoft.aspnetcore.authentication.jwtbearer.10.0.0.mupkg 188 ms
info : Se ha instalado Microsoft.AspNetCore.Authentication.JwtBearer 10.0.0 de https://api.nuget.org/v3/index.json a C:\Users\Iago.cadavidgonzalez\.nuget\packages\microsoft.aspnetcore.authentication.jwtbearer\10.0.0 con el hash de contenido 08gDf1GoZnzj30Bwx5vFWK53tq
sTEAs9pCwd1/0KxMUApfmeN6WdJof+CsmSd9tQquMedVdBo/QghuNTQ==.
info : GET https://api.nuget.org/v3/vulnerabilities/index.json 34 ms
info : OK https://api.nuget.org/v3/vulnerabilities/index.json 34 ms
info : GET https://api.nuget.org/v2-vulnerabilities/2025.11.19.23.31.42/vulnerability.base.json 45 ms
info : GET https://api.nuget.org/v3-vulnerabilities/2025.11.19.23.31.42/2025.11.24.23.31.59/vulnerability.update.json 82 ms
info : OK https://api.nuget.org/v3-vulnerabilities/2025.11.19.23.31.42/vulnerability.base.json 45 ms
info : OK https://api.nuget.org/v3-vulnerabilities/2025.11.19.23.31.42/2025.11.24.23.31.59/vulnerability.update.json 82 ms
error : NU1202: El paquete Microsoft.AspNetCore.Authentication.JwtBearer 10.0.0 no es compatible con net9.0 (.NETCoreApp,Version=v9.0). El paquete Microsoft.AspNetCore.Authentication.JwtBearer 10.0.0 admite: net10.0 (.NETCoreApp,Version=v10.0)
error : El paquete "Microsoft.AspNetCore.Authentication.JwtBearer" no es compatible con los marcos de trabajo de "all" del proyecto "C:\Users\Iago.cadavidgonzalez\Proyectos\reservas-app\backend\ReservasApi\ReservasApi.csproj".

C:\Users\Iago.cadavidgonzalez\Proyectos\reservas-app\backend\ReservasApi>
```

F

I've just opened cmd from ReservasApi folder (i.e. parent folder of ReservasApi.csproj file) and ran dotnet add package Microsoft.AspNetCore.Authentication.JwtBearer, but gott error: The package Microsoft.AspNetCore.Authentication.JwtBearer 10.0.0 is not compatible with net9.0

Ah! That error explains everything. In **.NET 9**, the versioning of NuGet packages changed slightly. The latest `Microsoft.AspNetCore.Authentication.JwtBearer` package **10.0.0** targets **.NET 10**, so it's **not compatible with net9.0**.

In **.NET 6–9**, you don't need to install a separate package — JWT authentication is already included in the shared framework (`Microsoft.AspNetCore.App`).

✔ What this means for your project

- 1. Do not run `dotnet add package Microsoft.AspNetCore.Authentication.JwtBearer` for **.NET 9**.
- 2. Just make sure you have these using statements in `Program.cs` :

```
csharp
using Microsoft.AspNetCore.Authentication.JwtBearer;
using Microsoft.IdentityModel.Tokens;
using System.Text;
```
- 3. The `builder.Services.AddAuthentication()` and `.AddJwtBearer(...)` code works without any extra NuGet package because it's part of the ASP.NET Core shared framework.

```

36 // Insert the authentication setup above "app.Build()", along with your other builder.Services.*
37 builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme).AddJwtBearer(/*...*/);
38 builder.Services.AddAuthorization();
39 builder.Services.AddScoped<AuthService>();
40

```

Lo tuve que poner “a

mano” entonces le dije a chatgpt que me lo chequera, me viene a decir que está bien pero que debo reemplazar `/*...*/` por algo más concreto:

1

```

builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme).AddJwtBearer(
    /*...*/);

```

✓ Correct structure, but:

- The `/*...*/` placeholder needs to be replaced with a lambda that configures `TokenValidationParameters`, e.g.:

csharp

Copy code

```

builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(options =>
    {
        options.TokenValidationParameters = new TokenValidationParameters
        {
            ValidateIssuer = false,
            ValidateAudience = false,
            ValidateIssuerSigningKey = true,
            IssuerSigningKey = new SymmetricSecurityKey(
                Encoding.UTF8.GetBytes(builder.Configuration["Jwt:Key"]))
        }
    });

```



This is **required** for JWT authentication to actually work.

NEXT PAGE

2 `builder.Services.AddAuthorization();`

✓ Correct.

- This registers the Authorization services in the DI container.
- Needed for `[Authorize]` attributes to function.

3 `builder.Services.AddScoped<AuthService>();`

✓ Correct.

- Registers your custom `AuthService` so it can be injected into controllers.
- `Scoped` is the right lifetime for services that access `DbContext`.

⚠ Important Notes

1. Make sure all **using statements** are present at the top of `Program.cs`:

csharp

```
using Microsoft.AspNetCore.Authentication.JwtBearer;  
using Microsoft.IdentityModel.Tokens;  
using System.Text;
```



Ahora mismo tengo además de los 3 using, esto:

```
36 // Insert the authentication setup above "app.Build()", along with your other builder.Services.*  
37 builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)  
38     .AddJwtBearer(opciones =>  
39     {  
40         opciones.TokenValidationParameters = new TokenValidationParameters // me autocompletó el primer TVP, no el segundo.  
41         {  
42             ValidateIssuer = false,  
43             ValidateAudience = false,  
44             ValidateIssuerSigningKey = true,  
45             IssuerSigningKey = new SymmetricSecurityKey(  
46                 Encoding.UTF8.GetBytes(builder.Configuration["Jwt:Key"])  
47             )  
48         };  
49     }); // lo que viene justo después de AddJwtBearer es una lambda that configures TokenValidationParameters.  
50 builder.Services.AddAuthorization();  
51 builder.Services.AddScoped<AuthService>();  
52  
53 // in ASP.NET Core (including .NET 9), all service registrations must be added before:  
54 var app = builder.Build(); // ya viene por defecto.
```

Según chatgpt my lambda code está bien y es válido para mi caso concreto, el cual se lo expliqué al detalle.

- The lambda goes **inside** `AddAuthentication(...).AddJwtBearer(...)` **before** `builder.Build()` .
- After `app = builder.Build()` , call:

csharp

 Copy

```
app.UseAuthentication();  
app.UseAuthorization();
```

✅ Conclusion

- Your code is **functionally correct**.

Anteriormente esto lo explicó así:

✓ PART 6 — Enable JWT Authentication in `Program.cs`

Add at top:

```
csharp

using Microsoft.AspNetCore.Authentication.JwtBearer;
using Microsoft.IdentityModel.Tokens;
using System.Text;
```

Add before `builder.Build()`:

```
csharp

builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(options =>
    {
        options.TokenValidationParameters = new TokenValidationParameters
        {
            ValidateIssuer = false,
            ValidateAudience = false,
            ValidateIssuerSigningKey = true,
            IssuerSigningKey = new SymmetricSecurityKey(
                Encoding.UTF8.GetBytes(builder.Configuration["Jwt:Key"]))
        }
    });

builder.Services.AddScoped<AuthService>();
```

Add after `app.UseRouting()`:

```
csharp

app.UseAuthentication();
app.UseAuthorization();
```

◆ Full corrected Program.cs for your backend

csharp Copy icon

```
using Microsoft.EntityFrameworkCore;
using ReservasApi.Data;
using Microsoft.AspNetCore.Authentication.JwtBearer;
using Microsoft.IdentityModel.Tokens;
using System.Text;

var builder = WebApplication.CreateBuilder(args);

// -----
// 1. Add services
// -----

builder.Services.AddControllers();

builder.Services.AddDbContext<AppDbContext>(options =>
    options.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection"))
);

builder.Services.AddEndpointsApiExplorer();
builder.Services.AddSwaggerGen();

// JWT Authentication
builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
    .AddJwtBearer(options =>
    {
        options.TokenValidationParameters = new TokenValidationParameters
        {
            ValidateIssuer = false,
            ValidateAudience = false,
            ValidateIssuerSigningKey = true,
            IssuerSigningKey = new SymmetricSecurityKey(
                Encoding.UTF8.GetBytes(builder.Configuration["Jwt:Key"])
            )
        };
    });

builder.Services.AddAuthorization();
builder.Services.AddScoped<AuthService>();

// -----
// 2. Build the app
// -----

var app = builder.Build();

// -----
// 3. Configure middleware
// -----

if (app.Environment.IsDevelopment())
{
    app.UseSwagger();
    app.UseSwaggerUI();
}

app.UseHttpsRedirection();

// Authentication must come before Authorization
app.UseAuthentication();
app.UseAuthorization();

// Map controllers (your /api/... endpoints)
app.MapControllers();

app.Run();
```



Lo hago yo:

```
Program.cs X ReservasApi.csproj
C: > Users > iago.cadavidgonzalez > Proyectos > reservas-app > backend > ReservasApi > Program.cs > Program > <top-level-statements-entry-point>
1  using Microsoft.EntityFrameworkCore; // aparece al poner UseSqlServer
2  using ReservasApi.Data;
3
4  // using ReservasApi.Eventos; // puse esta línea al poner con "Register the Kafka producer in DI (Dependency Injection)"
5
6  // para autenticación:
7  using Microsoft.AspNetCore.Authentication.JwtBearer; // no intellisense autocompletion
8  using Microsoft.IdentityModel.Tokens; // no intellisense autocompletion
9  using System.Text; // intellisense me acierta el Text.
10
11 var builder = WebApplication.CreateBuilder(args); // esto ya venía por defecto.
12
13 // 1. Add services to the container.
14
15 // Add controllers (this enables the [ApiController] controllers)
16 builder.Services.AddControllers(); //.AddOpenApi();
17
18 /*
19  .AddJsonOptions(opciones => // esto lo añadido con la intención de que get api reserva
20  {
21      opciones.JsonSerializerOptions.ReferenceHandler = System.Text.Json.Serialization.ReferenceHandler.IgnoreCycles;
22      opciones.JsonSerializerOptions.WriteIndented = true;
23  });
24  */
25
26 // Add EF Core and configure SQL Server using the connection string from appsettings.json
27 builder.Services.AddDbContext<AppDbContext>(opciones =>
28 {
29     opciones.UseSqlServer(builder.Configuration.GetConnectionString("DefaultConnection"));
30 });
31
32 // Add Swagger/OpenAPI support for testing your endpoints easily
33 builder.Services.AddEndpointsApiExplorer();
34 builder.Services.AddSwaggerGen(); // requires installation of the Swagger (Swashbuckle) NuGet package
35
36 // Register the Kafka producer in DI, para ello añadido esta línea antes de var app = builder.Build();
37 //builder.Services.AddSingleton<IServicioProductorKafka, ProductorKafka>(); // ProductorKafka
38 // COMENTO LA LÍNEA ANTERIOR DE CARA A EMPEZAR CON TESTS UNITARIOS IGNORAR TODO LO DE KAFKA
```

Sigue en next page

```

38 // Insert the authentication setup above "app.Build()", along with your other builder.Services.*
39 // JWT Authentication:
40 builder.Services.AddAuthentication(JwtBearerDefaults.AuthenticationScheme)
41 .AddJwtBearer(opciones =>
42 {
43     opciones.TokenValidationParameters = new TokenValidationParameters // me autocompletó el primer TVP, no el segundo.
44     {
45         ValidateIssuer = false,
46         ValidateAudience = false,
47         ValidateIssuerSigningKey = true,
48         IssuerSigningKey = new SymmetricSecurityKey(
49             Encoding.UTF8.GetBytes(builder.Configuration["Jwt:Key"])
50         )
51     };
52 }); // lo que viene justo después de AddJwtBearer es una lambda that configures TokenValidationParameters.
53 builder.Services.AddAuthorization();
54 builder.Services.AddScoped<AuthService>();
55

```

```

56 // 2. Build the app
57 // in ASP.NET Core (including .NET 9), all service registrations must be added before:
58 var app = builder.Build(); // ya viene por defecto. Esta línea es toda la sección 2. Build the app
59
60 // 2. Configure the HTTP request pipeline.
61 // A 25 de nov 2025 (al estar con lo de la autenticación), chatgpt a esto le llama 3. Configure middleware
62 if (app.Environment.IsDevelopment()) // esto ya venía por defecto
63 {
64     //app.MapOpenApi(); este es el único que venía por defecto.
65     // Enable Swagger UI only in development :
66     app.UseSwagger(); // esto lo puse yo
67     app.UseSwaggerUI(); // esto lo puse yo
68 }
69
70 app.UseHttpsRedirection(); // ya venía por defecto.
71
72 // Authentication must come before Authorization:
73 app.UseAuthentication();
74 app.UseAuthorization(); // lo puse yo.
75
76 // Map controller routes (this enables your /api/... endpoints)
77 app.MapControllers(); // lo puse yo.
78
79 app.Run(); // lo puse yo.

```

Regarding "Full corrected Program.cs for your backend", you did not write `app.UseRouting()`

Exactly — in your .NET 9 minimal hosting setup, you do not need `app.UseRouting()`. Here's why:

Añado el Jwt con la super secret key that is at least 32 chars long (conté que la que puse tiene 33 chars):


```
Program.cs X appsettings.json X ReservasApi.csproj
C: > Users > iago.cadavidgonzalez > Proyectos > reservas-app > backend > ReservasApi > {} appsettings.json > {} Jwt > abc Key
1  {
2    "ConnectionStrings": {
3      "DefaultConnection": "Server=db;Database=ReservasDb;User Id=sa;Password=docKer!8;TrustServerCertificate=True;"
4    },
5    "Jwt": {
6      "Key": "aP9d73D1jD0m3Ok1Lma17uiWe05vNfg57"
7    },
8    "Logging": {
9      "LogLevel": {
10       "Default": "Information",
11       "Microsoft.AspNetCore": "Warning"
12     }
13   },
14   "AllowedHosts": "*"
15 }
16
```

Y eso es este paso:

✓ PART 5 — Add JWT configuration in appsettings.json

json

```
"Jwt": {
  "Key": "some-super-secret-key-at-least-32-bytes-long"
}
```

In production, use environment variables or Docker secrets.

Otra cosa que según chatgpt debo tocar para lograr la implementación de autenticación básica es mi modelo de tabla de usuarios, que actualmente es así:

NEXT PAGE

```

1 using System.ComponentModel.DataAnnotations;
2
3 namespace ReservasApi.Modelos
4 {
5     7 references
6     public class Usuario
7     {
8         [Key]
9         5 references
10        public string Dni { get; set; } = string.Empty; // el dni es la primary key referenciada por fk en tabla de reservas
11
12        2 references
13        public string NombreCompleto { get; set; } = string.Empty;
14        0 references
15        public string Email { get; set; } = string.Empty;
16        0 references
17        public string Telefono { get; set; } = string.Empty;
18        0 references
19        public string Domicilio { get; set; } = string.Empty;
20
21        // fechas:
22        0 references
23        public DateOnly FechaNacimiento { get; set; }
24        0 references
25        public DateTime FechaRegistro { get; set; }
26
27        // contraseña:
28        [Required] // makes the property required, so that it cannot be null or omitted when inserting/updating.
29        0 references
30        public string Contraseña { get; set; } = string.Empty; // this is "the property declaration", above it pones la 'Required' keyword.
31
32        // propiedad de navegación: un usuario realiza entre 0 y varias reservas
33        0 references
34        public ICollection<Reserva>? Reservas { get; set; } // el cero lo represento con ? y el varios con ICollection.
35    } // final de public class Usuario
36 }

```

Sobre ese código, dice chatgpt lo siguiente: “your current ‘Usuario’ model mostly represents a **business user**” (with DNI, name, email, phone, address, reservations, etc.), which is fine for your booking app. **But for secure authentication, it’s not enough.**

1 Problem with your current model

csharp

```

[Required]
public string Contraseña { get; set; } = string.Empty;

```

- Right now you’re storing the **raw password** (`Contraseña`) as a string.
- **Security risk:** storing plain text passwords is **very unsafe**.
- For JWT-based authentication, you need:
 - **Password hash** (`byte[]`)
 - **Password salt** (`byte[]`)
- This allows you to **verify passwords without storing the raw password**. which is a security risk.

NEXT PAGE

2 How to modify your `Usuario` model for authentication

You have two options:

Option A — Separate authentication properties

Keep your business fields (DNI, Name, Email, etc.) and add:

```
csharp

[Required]
public byte[] PasswordHash { get; set; }

[Required]
public byte[] PasswordSalt { get; set; }
```

- Remove or ignore the plain `Contrasinal` property.
- Use `PasswordHash` and `PasswordSalt` in your login/registration logic.

```
/*
// contraseña as a raw string NO es segura:
[Required] // makes the property required, so that it cannot be null or omitted when inserting/updating.
public string Contrasinal { get; set; } = string.Empty; // this is "the property declaration", above it pones la 'Required'
*/

// Reemplazo la contraseña as raw string (no segura) por pwhash y pwsalt:
[Required]
0 references
public byte[] PasswordHash { get; set; }
[Required]
0 references
public byte[] PasswordSalt { get; set; }
```

Ahora bien, esos cambios supongo que me perturbarán los testeos con swagger y postman, ya que supongo que tendré que acordemente cambiar alguna otra cosilla por ahí.

NEXT PAGE

Lo que acabo de hacer es esto:

3 Recommended change (simplest)

Modify your model like this:

```
csharp Copy code

[Required]
public byte[] PasswordHash { get; set; }

[Required]
public byte[] PasswordSalt { get; set; }
```

- Remove this line:

```
csharp Copy code

public string Contraseña { get; set; } = string.Empty;
```

- Your existing fields (DNI, NombreCompleto, Email, etc.) can stay — they will be returned in APIs after login.

Y justo después deso dice esto (sin saber mi código actual al respecto):

4 Update DbContext & database

```
csharp

public DbSet<Usuario> Usuarios { get; set; }
```

Then:

```
bash

dotnet ef migrations add AddUsuarioAuth
dotnet ef database update
```

✓ PART 2 — Add DbSet in your DbContext

Open Data/ReservasContext.cs and add:

```
csharp

public DbSet<Usuario> Usuarios { get; set; }
```

Then run migration:

```
bash

dotnet ef migrations add AddUsuarios
dotnet ef database update
```

= (Dockerized SQL Server works fine, connection string must match.)

Regarding "Update DbContext" know that I already have a folder named Data (which is inside the folder named 'ReservasApi', which is inside the folder named 'backend'), and this folder named Data has just one thing inside, which is the a file named AppDbContext.cs, which has the following relevant code (with relevant I mean regarding Usuario):

```
public class AppDbContext : DbContext
{
    public DbSet<Usuario> Usuarios {get; set;}
}
```

Perfect — that's exactly what you need in your `AppDbContext` for Entity Framework to know about your `Usuario` table. A few notes and next steps regarding **authentication**:

1 Your DbContext is already correctly set up

csharp

 Copy code

```
public class AppDbContext : DbContext
{
    public DbSet<Usuario> Usuarios { get; set; }
}
```

- EF Core now knows to create a `Usuarios` table corresponding to your `Usuario` model.
- Once you add `PasswordHash` and `PasswordSalt` properties to `Usuario`, EF will detect the new columns in the next migration. 

Lo de add PasswordHash y PasswordSalt (y a cambio quitar la raw string pw) ya lo hice hace un rato.

NEXT PAGE

2 Next step: Update `Usuario` for authentication

As we discussed, modify your `Usuario` model to replace the plain `Contraseña` property with:

```
csharp

[Required]
public byte[] PasswordHash { get; set; }

[Required]
public byte[] PasswordSalt { get; set; }
```

- Keep all your other business fields (DNI, NombreCompleto, etc.) — they can coexist.

3 Create a new EF migration

After changing the `Usuario` model:

1. Open a terminal in the `ReservasApi` folder (where `ReservasApi.csproj` lives).
2. Run:

```
bash

dotnet ef migrations add AddUsuarioAuth
dotnet ef database update
```

- This will:
 - Add `PasswordHash` and `PasswordSalt` columns to the `Usuarios` table.
 - Keep your existing columns (DNI, name, etc.) intact.

✓ Your database will now be ready for secure user registration/login.

De momento no voy a hacer las migraciones esas de actualizar la bd.

De los pasos que me dijo para implement basic authentication, estos son los hechos y los no hechos por mí hasta ahora:

1. Create a “User” model poniendo **pwhash** y **pwsalt** en lugar de raw string pw (esto pasa para `DTORregistro` y `DTOlogin`).
2. Add `DbSet<Usuario>` in your `AppDbContext` (ya lo tenía). **Run migrations to update db.**
3. Create **DTOs for Register/Login**
4. Create a **service for pw hashing + JWT token generation** i.e. create a folder `Services` with file **`AuthService.cs`**.

5. **Add JWT configuration in appsettings.json** (del backend)
6. **Enable JWT Authentication in Program.cs** (del backend)
7. Create an **AuthenticationController**
8. **Protect** your existing **CRUD Controllers**
9. **Test in Swagger**

3 Create a new EF migration

After changing the `Usuario` model:

1. Open a terminal in the `ReservasApi` folder (where `ReservasApi.csproj` lives).
2. Run:

```
bash

dotnet ef migrations add AddUsuarioAuth
dotnet ef database update
```

- This will:
 - Add `PasswordHash` and `PasswordSalt` columns to the `Usuarios` table.
 - Keep your existing columns (DNI, name, etc.) intact.

✓ Your database will now be ready for secure user registration/login.

4 What comes next

Once the database is updated:



1. **Create DTOs for** registration and login (so you don't send raw hashes over the API).

Voy a **run migrations** pero siendo consciente de que seguramente me perturba los swagger/postman tests al no cambiar otras cosas de mi código.

NEXT PAGE

Before running:
dotnet ef migrations add AddUsuarioAuth
dotnet ef database update

Should I stop backend docker container and sql server docker container ?

No — you **do not need to stop your Docker containers** to run EF Core migrations. In fact, you usually **run migrations against the running database**.

Here's the correct approach:

1 Backend container

- You **don't need to stop it**.
- The migrations are applied directly from your local `dotnet ef` command, connecting to the SQL Server in your container.

2 SQL Server container

- The container **must be running**, because EF Core needs to connect to the database.
- If it's stopped, migrations will fail because there's no database to connect to.

Pasa que el dotnet ef migrations add AddUsuarioAuth no va porque falla dotnet build:

```
C:\Users\iago.cadavidgonzalez\Proyectos\reservas-app\backend\ReservasApi>dotnet ef migrations AddUsuarioAuth
Build started...
Build failed. Use dotnet build to see the errors.

C:\Users\iago.cadavidgonzalez\Proyectos\reservas-app\backend\ReservasApi>dotnet build
Restauración completada (0,3s)
ReservasApi error con 1 errores (0,5s)
  C:\Users\iago.cadavidgonzalez\Proyectos\reservas-app\backend\ReservasApi\Program.cs(7,43): error CS0234: El tipo o el nombre del espacio de nombres 'JwtBearer' no existe en el espacio de nombres 'Microsoft.AspNetCore.Authentication' (¿falta alguna referencia de ensamblado?)
Compilación error con 1 errores en 1,4s
```

Option A — Install the compatible NuGet package

- For .NET 9, the package you need is `Microsoft.AspNetCore.Authentication.JwtBearer` version 9.x, not 10.x.

```
bash
```

[Copy code](#)

```
dotnet add package Microsoft.AspNetCore.Authentication.JwtBearer --version 9.0.0
```

- Then rebuild:

```
bash
```

[Copy code](#)

```
dotnet build
```

✅ This will make the namespace `Microsoft.AspNetCore.Authentication.JwtBearer` available.

```
C:\Users\iago.cadavidgonzalez\Proyectos\reservas-app\backend\ReservasApi>dotnet add package Microsoft.AspNetCore.Authentication.JwtBearer --version 9.0.0

Compilación realizada correctamente en 0,6s
info : La validación de la cadena de certificados X.509 utilizará el almacén de confianza predeterminado seleccionado por .NET para la firma de código.
info : La validación de la cadena de certificados X.509 utilizará el almacén de confianza predeterminado seleccionado por .NET para la marca de tiempo.
info : Agregando PackageReference para el paquete "Microsoft.AspNetCore.Authentication.JwtBearer" al proyecto "C:\Users\iago.cadavidgonzalez\Proyectos\reservas-app\backen
d\ReservasApi\ReservasApi.csproj".
info : Restaurando paquetes para C:\Users\iago.cadavidgonzalez\Proyectos\reservas-app\backend\ReservasApi\ReservasApi.csproj...
info :   GET https://api.nuget.org/v3-flatcontainer/microsoft.aspnetcore.authentication.jwtbearer/index.json
info :   OK https://api.nuget.org/v3-flatcontainer/microsoft.aspnetcore.authentication.jwtbearer/index.json 387 ms
info :   GET https://api.nuget.org/v3-flatcontainer/microsoft.aspnetcore.authentication.jwtbearer/9.0.0/microsoft.aspnetcore.authentication.jwtbearer.9.0.0.nupkg
info :   OK https://api.nuget.org/v3-flatcontainer/microsoft.aspnetcore.authentication.jwtbearer/9.0.0/microsoft.aspnetcore.authentication.jwtbearer.9.0.0.nupkg 106 ms
info :   GET https://api.nuget.org/v3-flatcontainer/microsoft.identitymodel.protocols.openidconnect/index.json
info :   OK https://api.nuget.org/v3-flatcontainer/microsoft.identitymodel.protocols.openidconnect/index.json 156 ms
info :   GET https://api.nuget.org/v3-flatcontainer/microsoft.identitymodel.protocols.openidconnect/8.0.1/microsoft.identitymodel.protocols.openidconnect.8.0.1.nupkg
info :   OK https://api.nuget.org/v3-flatcontainer/microsoft.identitymodel.protocols.openidconnect/8.0.1/microsoft.identitymodel.protocols.openidconnect.8.0.1.nupkg 51 ms
info :   GET https://api.nuget.org/v3-flatcontainer/microsoft.identitymodel.protocols/index.json
info :   GET https://api.nuget.org/v3-flatcontainer/system.identitymodel.tokens.jwt/index.json
info :   OK https://api.nuget.org/v3-flatcontainer/system.identitymodel.tokens.jwt/index.json 240 ms
info :   GET https://api.nuget.org/v3-flatcontainer/system.identitymodel.tokens.jwt/8.0.1/system.identitymodel.tokens.jwt.8.0.1.nupkg
info :   OK https://api.nuget.org/v3-flatcontainer/system.identitymodel.tokens.jwt/8.0.1/system.identitymodel.tokens.jwt.8.0.1.nupkg 50 ms
info :   OK https://api.nuget.org/v3-flatcontainer/microsoft.identitymodel.protocols/index.json 427 ms
info :   GET https://api.nuget.org/v3-flatcontainer/microsoft.identitymodel.protocols/8.0.1/microsoft.identitymodel.protocols.8.0.1.nupkg
info :   OK https://api.nuget.org/v3-flatcontainer/microsoft.identitymodel.protocols/8.0.1/microsoft.identitymodel.protocols.8.0.1.nupkg 47 ms
info :   GET https://api.nuget.org/v3-flatcontainer/microsoft.identitymodel.tokens/index.json
info :   GET https://api.nuget.org/v3-flatcontainer/microsoft.identitymodel.jsonwebtokens/index.json
info :   OK https://api.nuget.org/v3-flatcontainer/microsoft.identitymodel.tokens/index.json 149 ms
info :   OK https://api.nuget.org/v3-flatcontainer/microsoft.identitymodel.jsonwebtokens/index.json 162 ms
info :   GET https://api.nuget.org/v3-flatcontainer/microsoft.identitymodel.tokens/8.0.1/microsoft.identitymodel.tokens.8.0.1.nupkg
info :   GET https://api.nuget.org/v3-flatcontainer/microsoft.identitymodel.jsonwebtokens/8.0.1/microsoft.identitymodel.jsonwebtokens.8.0.1.nupkg
info :   OK https://api.nuget.org/v3-flatcontainer/microsoft.identitymodel.tokens/8.0.1/microsoft.identitymodel.tokens.8.0.1.nupkg 39 ms
info :   OK https://api.nuget.org/v3-flatcontainer/microsoft.identitymodel.jsonwebtokens/8.0.1/microsoft.identitymodel.jsonwebtokens.8.0.1.nupkg 48 ms
info :   GET https://api.nuget.org/v3-flatcontainer/microsoft.identitymodel.logging/index.json
info :   OK https://api.nuget.org/v3-flatcontainer/microsoft.identitymodel.logging/index.json 176 ms
info :   GET https://api.nuget.org/v3-flatcontainer/microsoft.identitymodel.logging/8.0.1/microsoft.identitymodel.logging.8.0.1.nupkg
info :   OK https://api.nuget.org/v3-flatcontainer/microsoft.identitymodel.logging/8.0.1/microsoft.identitymodel.logging.8.0.1.nupkg 37 ms
info :   GET https://api.nuget.org/v3-flatcontainer/microsoft.identitymodel.abstractions/index.json
info :   OK https://api.nuget.org/v3-flatcontainer/microsoft.identitymodel.abstractions/index.json 168 ms
info :   GET https://api.nuget.org/v3-flatcontainer/microsoft.identitymodel.abstractions/8.0.1/microsoft.identitymodel.abstractions.8.0.1.nupkg
info :   OK https://api.nuget.org/v3-flatcontainer/microsoft.identitymodel.abstractions/8.0.1/microsoft.identitymodel.abstractions.8.0.1.nupkg 41 ms
info : Se ha instalado Microsoft.AspNetCore.Authentication.JwtBearer 9.0.0 de https://api.nuget.org/v3/index.json a C:\Users\iago.cadavidgonzalez\.nuget\packages\microsof
t.aspnetcore.authentication.jwtbearer\9.0.0 con el hash de contenido bs+1Pg3vQd52lTyxNUd9fEhtM5q3eLUpK36k2t56iDwVrk60rAoFtVrQrTK0Y00etTc3rUKGU7hBlf+0RzHLqQ==.
info : Se ha instalado Microsoft.IdentityModel.Protocols.OpenIdConnect 8.0.1 de https://api.nuget.org/v3/index.json a C:\Users\iago.cadavidgonzalez\.nuget\packages\micros
oft.identitymodel.protocols.openidconnect\8.0.1 con el hash de contenido AQDbfPLyzuuGh0/mQHkNsp4pm5Jv8/BI4K1FXR7beVGZoSH35zHV3PrmcfvSTsyI6qrCR898NzUauD6SR1gg==.
```

Finaliza en next page

```
info : Se ha instalado Microsoft.IdentityModel.Logging 8.0.1 de https://api.nuget.org/v3/index.json a C:\Users\iago.cadavidgonzalez\.nuget\packages\microsoft.identitymodel.logging\8.0.1 con el hash de contenido UCPF2exZq8Xe7v/6sGniM6zCQ0uXXQ9+v5VTb9gPB8ZSUPnX53BxLN78v2jsbIvK9Dq4GovQxo23x8JgWwm/Qg==.
info : Se ha instalado System.IdentityModel.Tokens.Jwt 8.0.1 de https://api.nuget.org/v3/index.json a C:\Users\iago.cadavidgonzalez\.nuget\packages\system.identitymodel.tokens.jwt\8.0.1 con el hash de contenido GJw3bYkWP0gvN3tJo5X4LYuIFA2HD293FPUhKmp7qxS+g5ywAb34Dnd3cDAFLkcMohy5XTpaaZ4uAHuw0uSPQ==.
info : Se ha instalado Microsoft.IdentityModel.Abstractions 8.0.1 de https://api.nuget.org/v3/index.json a C:\Users\iago.cadavidgonzalez\.nuget\packages\microsoft.identitymodel.abstractions\8.0.1 con el hash de contenido OtlIwcyX0i0lfdevPKZdIPf8EvbcioDyBiE/Z2lHsopsMD7twcKtLN9kMevHmI5IIPhFpfwCIIR6qHQz1WHUIw==.
info : Se ha instalado Microsoft.IdentityModel.Protocols 8.0.1 de https://api.nuget.org/v3/index.json a C:\Users\iago.cadavidgonzalez\.nuget\packages\microsoft.identitymodel.protocols\8.0.1 con el hash de contenido uA2vpKqU3I2mBBEaeJAWPTjT9v1TzrGWKdgK6G5qJd03CLx83kdiq09cmiK8/nlerkH2FBWU/RphP83aAe3i3g==.
info : Se ha instalado Microsoft.IdentityModel.JsonWebTokens 8.0.1 de https://api.nuget.org/v3/index.json a C:\Users\iago.cadavidgonzalez\.nuget\packages\microsoft.identitymodel.jsonwebtokens\8.0.1 con el hash de contenido s6++gF9x0rQApQz0BbSyp4jUaAlwm+DroKfL8gd0Hxs83k8SJfUXhuc46rDB3rNXBQ1MVRxqKUrqFh0/M0E97g==.
info : Se ha instalado Microsoft.IdentityModel.Tokens 8.0.1 de https://api.nuget.org/v3/index.json a C:\Users\iago.cadavidgonzalez\.nuget\packages\microsoft.identitymodel.tokens\8.0.1 con el hash de contenido kDimB6Dkd3nkW2oZPDkMkVhfQt3IDqO5gL0oa8WVvY30P4uE8Ij+8TXnqg9T0d9ufjsY3IDiGz7pCubnfl18tjg==.
info : GET https://api.nuget.org/v3/vulnerabilities/index.json
info : OK https://api.nuget.org/v3/vulnerabilities/index.json 46 ms
info : GET https://api.nuget.org/v3-vulnerabilities/2025.11.19.23.31.42/vulnerability.base.json
info : GET https://api.nuget.org/v3-vulnerabilities/2025.11.19.23.31.42/2025.11.25.05.32.00/vulnerability.update.json
info : OK https://api.nuget.org/v3-vulnerabilities/2025.11.19.23.31.42/vulnerability.base.json 45 ms
info : OK https://api.nuget.org/v3-vulnerabilities/2025.11.19.23.31.42/2025.11.25.05.32.00/vulnerability.update.json 54 ms
info : El paquete "Microsoft.AspNetCore.Authentication.JwtBearer" es compatible con todos los marcos de trabajo especificados del proyecto "C:\Users\iago.cadavidgonzalez\Proyectos\reservas-app\backend\ReservasApi\ReservasApi.csproj".
info : Se agregó PackageReference para la versión "9.0.0" del paquete "Microsoft.AspNetCore.Authentication.JwtBearer" al archivo "C:\Users\iago.cadavidgonzalez\Proyectos\reservas-app\backend\ReservasApi\ReservasApi.csproj".
info : Generación de archivo MSBuild C:\Users\iago.cadavidgonzalez\Proyectos\reservas-app\backend\ReservasApi\obj\ReservasApi.csproj.nuget.g.props.
info : Escribiendo el archivo de recursos en el disco. Ruta de acceso: C:\Users\iago.cadavidgonzalez\Proyectos\reservas-app\backend\ReservasApi\obj\project.assets.json
log : Se ha restaurado C:\Users\iago.cadavidgonzalez\Proyectos\reservas-app\backend\ReservasApi\ReservasApi.csproj (en 4,21 s).

C:\Users\iago.cadavidgonzalez\Proyectos\reservas-app\backend\ReservasApi>
```

Ahora aparece JwtBearer en el csproj:

```
Program.cs X ReservasApi.csproj X appsettings.json

C: > Users > iago.cadavidgonzalez > Proyectos > reservas-app > backend > ReservasApi > ReservasApi.csproj

1  <Project Sdk="Microsoft.NET.Sdk.Web">
2
3  <PropertyGroup>
4  <TargetFramework>net9.0</TargetFramework>
5  <Nullable>enable</Nullable>
6  <ImplicitUsings>enable</ImplicitUsings>
7  </PropertyGroup>
8
9  <ItemGroup>
10 <PackageReference Include="Confluent.Kafka" Version="2.12.0" />
11 <PackageReference Include="Microsoft.AspNetCore.Authentication.JwtBearer" Version="9.0.0" />
12 <PackageReference Include="Microsoft.AspNetCore.OpenApi" Version="9.0.10" />
13 <PackageReference Include="Microsoft.EntityFrameworkCore" Version="9.0.10" />
14 <PackageReference Include="Microsoft.EntityFrameworkCore.Design" Version="9.0.10">
15 <IncludeAssets>runtime; build; native; contentfiles; analyzers; buildtransitive</IncludeAssets>
16 <PrivateAssets>all</PrivateAssets>
17 </PackageReference>
18 <PackageReference Include="Microsoft.EntityFrameworkCore.SqlServer" Version="9.0.10" />
19 <PackageReference Include="Microsoft.EntityFrameworkCore.Tools" Version="9.0.10">
20 <PrivateAssets>all</PrivateAssets>
21 <IncludeAssets>runtime; build; native; contentfiles; analyzers; buildtransitive</IncludeAssets>
22 </PackageReference>
23 <PackageReference Include="Swashbuckle.AspNetCore" Version="9.0.6" />
24 </ItemGroup>
25
26 </Project>
27
```

Para quitar las dos alertas sobre pwhash y pwsalt simplemente pongo byte[]? en lugar de byte[], entonces vuelvo a hacer dotnet build:

```
C:\Users\iago.cadavidgonzalez\Proyectos\reservas-app\backend\ReservasApi>dotnet build
Restauración completada (0,3s)
ReservasApi: error con 1 errores y 1 advertencias (0,7s)
C:\Users\iago.cadavidgonzalez\Proyectos\reservas-app\backend\ReservasApi\Program.cs(54,28): error CS0246: El nombre del tipo o del espacio de nombres 'AuthService' no se encontró (¿falta una directiva using o una referencia de ensamblado?)
C:\Users\iago.cadavidgonzalez\Proyectos\reservas-app\backend\ReservasApi\Program.cs(49,40): warning CS8604: Posible argumento de referencia nulo para el parámetro "s" en "byte[] Encoding.GetBytes(string s)".

Compilación error con 1 errores y 1 advertencias en 1,6s

C:\Users\iago.cadavidgonzalez\Proyectos\reservas-app\backend\ReservasApi>
```

2 Error: AuthService not found (CS0246)

powershell

error CS0246: El nombre del tipo o del espacio de nombres 'AuthService' no se encontró

Why:

- You haven't created the `AuthService` class yet.
- In your `Program.cs` you wrote:

csharp

```
builder.Services.AddScoped<AuthService>();
```

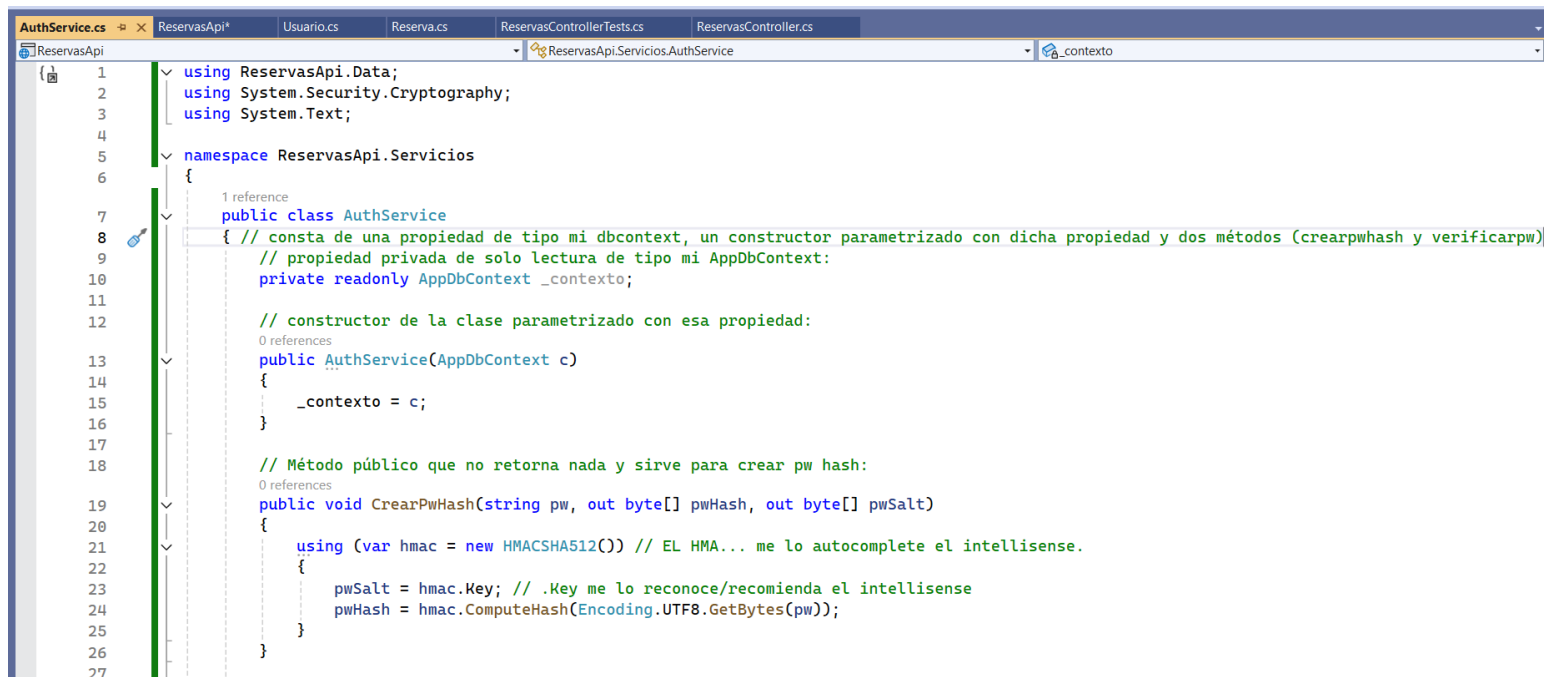
- This fails because there's no such class defined.

Fix:

1. Create a new file `Services/AuthService.cs`:

Osease

4. Create a service for pw hashing + JWT token generation i.e. create a folder `Services` with file `AuthService.cs`.¶



```
AuthService.cs | ReservasApi* | Usuario.cs | Reserva.cs | ReservasControllerTests.cs | ReservasController.cs
ReservasApi | ReservasApi.Servicios.AuthService | _contexto

1  using ReservasApi.Data;
2  using System.Security.Cryptography;
3  using System.Text;
4
5  namespace ReservasApi.Servicios
6  {
7      1 reference
8      public class AuthService
9      { // consta de una propiedad de tipo mi dbcontext, un constructor parametrizado con dicha propiedad y dos métodos (crearpwhash y verificarpw)
10         // propiedad privada de solo lectura de tipo mi AppDbContext:
11         private readonly AppDbContext _contexto;
12
13         // constructor de la clase parametrizado con esa propiedad:
14         0 references
15         public AuthService(AppDbContext c)
16         {
17             _contexto = c;
18         }
19
20         // Método público que no retorna nada y sirve para crear pw hash:
21         0 references
22         public void CreaPwHash(string pw, out byte[] pwHash, out byte[] pwSalt)
23         {
24             using (var hmac = new HMACSHA512()) // EL HMA... me lo autocompleta el intellisense.
25             {
26                 pwSalt = hmac.Key; // .Key me lo reconoce/recomienda el intellisense
27                 pwHash = hmac.ComputeHash(Encoding.UTF8.GetBytes(pw));
28             }
29         }
30     }
31 }
```



```

28 // Método que retorna buleano y verifica la password:
29 0 references
30 public bool VerificarPwHash(string pw, byte[] hashGuardado, byte[] saltGuardado)
31 {
32     using (var hmac = new HMACSHA512(saltGuardado))
33     {
34         var hashComputado = hmac.ComputeHash(Encoding.UTF8.GetBytes(pw));
35         return hashComputado.SequenceEqual(hashGuardado);
36     } // final del método VerificarPwHash
37 } // final de la public class AuthService, la cual uso en Program.cs en línea builder.Services.AddScoped<AuthService>();
38 } // namespace
39
40

```

Your AuthService implementation is correct, safe and fully functional.

Necesario añadir al comienzo de Program.cs:

```

10 using ReservasApi.Servicios;

```

De lo contrario del dotnet build seguirá dando el mismo error de que no reconoce AuthService como si todavía no existiese.

Una vez añadio, ese using, vuelvo a hacer dotnet build:

```

C:\Users\iago.cadavidgonzalez\Proyectos\reservas-app\backend\ReservasApi>dotnet build
Restauración completada (0,3s)
ReservasApi correcto con 1 advertencias (8,1s) -> bin\Debug\net9.0\ReservasApi.dll
C:\Users\iago.cadavidgonzalez\Proyectos\reservas-app\backend\ReservasApi\Program.cs(50,40): warning CS8604: Posible argumento de referencia nulo para el parámetro "s" en "byte[] Encoding.GetBytes(string s)".
Compilación correcta con 1 advertencias en 9,0s

```

“The compiler warning does NOT mean the value is null; it only means the compiler cannot verify that it won’t be null.”

Según chatgpt, si añado

```

?? throw new Exception("Missing Jwt:Key in configuration")

```

En la línea de Program.cs que da la advertencia al correr el dotnet build, entonces me aseguro de que no saltará dicha advertencia y al mismo tiempo como en appsettings.json sí tengo lo que tengo que tener al respecto de la jwt:key, tampoco caeré en esa Exception y por ende no me dará ese mensaje sino que caeré en el caso de la izquierda del ??.

En concreto la cosa queda así:

```

IssuerSigningKey = new SymmetricSecurityKey(
    Encoding.UTF8.GetBytes(
        builder.Configuration["Jwt:Key"]
        ?? throw new Exception("Missing Jwt:Key in configuration")
    )
)

```

Y entonces:

```
C:\Users\iago.cadavidgonzalez\Proyectos\reservas-app\backend\ReservasApi>dotnet build
Restauración completada (0,3s)
ReservasApi realizado correctamente (1,2s) → bin\Debug\net9.0\ReservasApi.dll

Compilación realizado correctamente en 2,0s

C:\Users\iago.cadavidgonzalez\Proyectos\reservas-app\backend\ReservasApi>
```

Recordemos que todo esto de implementar el AuthService lo hice porque no podía hacer dotnet build y por ende tampoco el dotnet ef migrations add y por ende no me podía poner con los dtos:

1. Open a terminal in the `ReservasApi` folder (where `ReservasApi.csproj` lives).

2. Run:

```
bash

dotnet ef migrations add AddUsuarioAuth
dotnet ef database update
```

- This will:

- Add `PasswordHash` and `PasswordSalt` columns to the `Usuarios` table.
- Keep your existing columns (DNI, name, etc.) intact.

✓ Your database will now be ready for secure user registration/login.

4 What comes next

Once the database is updated:



1. **Create DTOs for** registration and login (so you don't send raw hashes over the API).

Hago ahora el dotnet ef migrations add AddUsuarioAuth (ya que ahora el dotnet build va perfecto)

```
C:\Users\iago.cadavidgonzalez\Proyectos\reservas-app\backend\ReservasApi>dotnet build
Restauración completada (0,3s)
ReservasApi realizado correctamente (1,2s) → bin\Debug\net9.0\ReservasApi.dll

Compilación realizado correctamente en 2,0s

C:\Users\iago.cadavidgonzalez\Proyectos\reservas-app\backend\ReservasApi>dotnet ef migrations add AddUsuarioAuth
Build started...
Build succeeded.
An operation was scaffolded that may result in the loss of data. Please review the migration for accuracy.
Done. To undo this action, use 'ef migrations remove'

C:\Users\iago.cadavidgonzalez\Proyectos\reservas-app\backend\ReservasApi>
```



```

C:\Users\iago.cadavidgonzalez\Proyectos\reservas-app\backend\ReservasApi>dotnet ef database update
Build started...
Build succeeded.
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (14ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      SELECT 1
info: Microsoft.EntityFrameworkCore.Migrations[20411]
      Acquiring an exclusive lock for migration application. See https://aka.ms/efcore-docs-migrations-lock for more information if this takes too long.
      Acquiring an exclusive lock for migration application. See https://aka.ms/efcore-docs-migrations-lock for more information if this takes too long.
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (23ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      DECLARE @result int;
      EXEC @result = sp_getapplock @Resource = '__EFMigrationsLock', @LockOwner = 'Session', @LockMode = 'Exclusive';
      SELECT @result
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (3ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      IF OBJECT_ID(N'[__EFMigrationsHistory]') IS NULL
      BEGIN
          CREATE TABLE [__EFMigrationsHistory] (
              [MigrationId] nvarchar(150) NOT NULL,
              [ProductVersion] nvarchar(32) NOT NULL,
              CONSTRAINT [PK__EFMigrationsHistory] PRIMARY KEY ([MigrationId])
          );
      END;
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (1ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      SELECT 1
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (2ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      SELECT OBJECT_ID(N'[__EFMigrationsHistory]');
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (43ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      SELECT [MigrationId], [ProductVersion]
      FROM [__EFMigrationsHistory]
      ORDER BY [MigrationId];
info: Microsoft.EntityFrameworkCore.Migrations[20402]
      Applying migration '20251125125516_AddUsuarioAuth'.
      Applying migration '20251125125516_AddUsuarioAuth'.
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (66ms) [Parameters=[], CommandType='Text', CommandTimeout='30']

```

```

info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (66ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      DECLARE @var sysname;
      SELECT @var = [d].[name]
      FROM [sys].[default_constraints] [d]
      INNER JOIN [sys].[columns] [c] ON [d].[parent_column_id] = [c].[column_id] AND [d].[parent_object_id] = [c].[object_id]
      WHERE ([d].[parent_object_id] = OBJECT_ID(N'[Usuarios]') AND [c].[name] = N'Contrasinal');
      IF @var IS NOT NULL EXEC(N'ALTER TABLE [Usuarios] DROP CONSTRAINT [' + @var + '];');
      ALTER TABLE [Usuarios] DROP COLUMN [Contrasinal];
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (21ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      ALTER TABLE [Usuarios] ADD [PasswordHash] varbinary(max) NOT NULL DEFAULT 0x;
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (4ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      ALTER TABLE [Usuarios] ADD [PasswordSalt] varbinary(max) NOT NULL DEFAULT 0x;
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (4ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      INSERT INTO [__EFMigrationsHistory] ([MigrationId], [ProductVersion])
      VALUES (N'20251125125516_AddUsuarioAuth', N'9.0.10');
info: Microsoft.EntityFrameworkCore.Database.Command[20101]
      Executed DbCommand (2ms) [Parameters=[], CommandType='Text', CommandTimeout='30']
      DECLARE @result int;
      EXEC @result = sp_releaseapplock @Resource = '__EFMigrationsLock', @LockOwner = 'Session';
      SELECT @result
Done.

C:\Users\iago.cadavidgonzalez\Proyectos\reservas-app\backend\ReservasApi>

```

Tuve que cambiar de 'db' a 'localhost,1433' en appsettings.json del backend para que ese comando no diera un error al final.

EF Core did not create a new database because the existing one (ReservasDb) already exists. Instead, it **updated your existing database** with the new table/columns. Each **migration** is a set of instructions (C# code) that describes how to **update a database schema**.

4 What comes next

Once the database is updated:



1. **Create DTOs for** registration and login (so you don't send raw hashes over the API).

NEXT PAGE

Is "creating the Register and Login endpoints with JWT so your basic authentication is fully functional." the same as "Create DTOs for registration and login (so you don't send raw hashes over the API)" ?

Yes — those two steps are closely related, but not exactly the same. Let me explain carefully.

1 Register and Login endpoints with JWT

This refers to the actual API methods you expose in a controller:

- `POST /api/auth/register` → creates a new user in the database
- `POST /api/auth/login` → checks credentials and returns a JWT token

These endpoints handle the logic: hashing passwords, verifying hashes, generating JWTs, etc.

2 DTOs for registration and login

DTOs = Data Transfer Objects — small classes that define exactly what data the API receives or returns.

Example:

```
csharp Copy code

public class RegisterDto
{
    public string Dni { get; set; } = string.Empty;
    public string NombreCompleto { get; set; } = string.Empty;
    public string Email { get; set; } = string.Empty;
    public string Password { get; set; } = string.Empty; // plain text from client
}

public class LoginDto
{
    public string Dni { get; set; } = string.Empty;
    public string Password { get; set; } = string.Empty; // plain text from client
}
```

Why you need DTOs:

- You never send raw `PasswordHash` or `PasswordSalt` over the API.
- You never receive `PasswordHash` or `PasswordSalt` directly from the client.
- DTOs let you send/receive only the fields you care about (like `Password` in plain text), then your `AuthService` hashes it internally.



What I wanna do now is to “**create DTOs for registration and login** (so I don’t send raw hashes over the API).

These **DTOs** are gonna be simple **C# classes** that **define exactly what data the client will send to your API**, so you don't expose PasswordHash or PasswordSalt over the network (supongo que se refiere a las propiedades de Usuario.cs).

I already have a folder called 'DTO', but I have it inside 'Modelos' folder, which is inside 'ReservasApi' folder.

That works perfectly — you don't need a separate top-level `DTOs` folder. 

You can keep your DTOs inside:

markdown

 Copy code

```
ReservasApi/  
└─ Modelos/  
    └─ DTO/
```

Just make sure that:

1. The **namespace matches the folder structure**, for example:

csharp

 Copy code

```
namespace ReservasApi.Modelos.DTO  
{  
    public class RegisterDto  
    {  
        // ...  
    }  
}
```

2. When you use them in your controller, you add a `using`:

csharp

 Copy code

```
using ReservasApi.Modelos.DTO;
```



Does RegisterDto replace UsuarioDto ? La respuesta es no:

NEXT PAGE

Not exactly — `RegisterDto` does not replace `UsuarioDto`; they serve different purposes.

Here's the distinction:

1 UsuarioDto

- Represents user data that you send to clients (read-only or safe information).
- Typically does not include passwords.
- Example: returning user details after registration or in a GET `/users` endpoint.

```
csharp Copy code

namespace ReservasApi.Modelos.DTO
{
    public class UsuarioDto
    {
        public string Dni { get; set; } = string.Empty;
        public string NombreCompleto { get; set; } = string.Empty;
        // Other fields you want the client to see
    }
}
```

2 RegisterDto

- Represents data the client sends to register a new user.
- Includes the plain-text password (which will be hashed server-side).
- Example: POST `/api/auth/register` endpoint.

```
csharp Copy code

namespace ReservasApi.Modelos.DTO
{
    public class RegisterDto
    {
        public string Dni { get; set; } = string.Empty;
        public string NombreCompleto { get; set; } = string.Empty;
        public string Email { get; set; } = string.Empty;
        public string Telefono { get; set; } = string.Empty;
        public string Domicilio { get; set; } = string.Empty;
        public DateOnly FechaNacimiento { get; set; }

        public string Password { get; set; } = string.Empty; // plain text from client
    }
}
```

RegisterDto.cs:

NEXT PAGE

RegisterDto.cs:

```
1 namespace ReservasApi.Modelos.DTO
2 {
3     0 references
4     public class RegisterDto
5     {
6         0 references
7         public string Dni { get; set; } = string.Empty; // igual que en Usuario.cs
8         0 references
9         public string NombreCompleto { get; set; } = string.Empty; // igual que en Usuario.cs
10        0 references
11        public string Email { get; set; } = string.Empty; // igual que en Usuario.cs
12        0 references
13        public string Telefono { get; set; } = string.Empty; // igual que en Usuario.cs
14        0 references
15        public string Domicilio { get; set; } = string.Empty; // igual que en Usuario.cs
16        0 references
17        public DateTime FechaNacimiento { get; set; } // igual que en Usuario.cs
18        0 references
19        public string Password { get; set; } = string.Empty; // plain text from client
20    } // final de la clase RegisterDto
21 }
```

I've just written RegisterDto, do I need to write any other Dto for our purpose of implementing basic authentication ?

Yes — for basic authentication you typically need **one more DTO** for login.

Here's why and what it looks like:

1 LoginDto

- Purpose: the client sends **credentials** to log in.
- Only needs **identifier** (DNI in your case) and **password**.

NEXT PAGE

Example:

csharp

```
namespace ReservasApi.Modelos.DTO
{
    public class LoginDto
    {
        public string Dni { get; set; } = string.Empty;
        public string Password { get; set; } = string.Empty;
    }
}
```

- You do **not** send `PasswordHash` or `PasswordSalt` from the client.
- The API uses your `AuthService` to **verify the password** and issue a JWT. *The client does not send, diría yo en 1er punto.*

So the following is enough for my LoginDto ?

```
namespace ReservasApi.Modelos.DTO
{
    public class LoginDto
    {
        public string Dni { get; set; } = string.Empty;
        public string Password { get; set; } = string.Empty;
    }
}
```

Yes — that is **perfect and sufficient** for your LoginDto. ✅

- It only includes the **fields the client needs to send**.
- It does **not expose** `PasswordHash` or `PasswordSalt`, which is exactly what you want.
- The `AuthService` will take care of verifying the password using the hash and salt stored in your database.

With `RegisterDto` and `LoginDto` in place, you now have all the input DTOs needed for basic authentication.

The next step is usually to create the **AuthController** that uses these DTOs along with `AuthService` to handle registration and login and generate JWT tokens.

De los pasos que me dijo para implement basic authentication, estos son los hechos y los no hechos por mí hasta ahora:¶

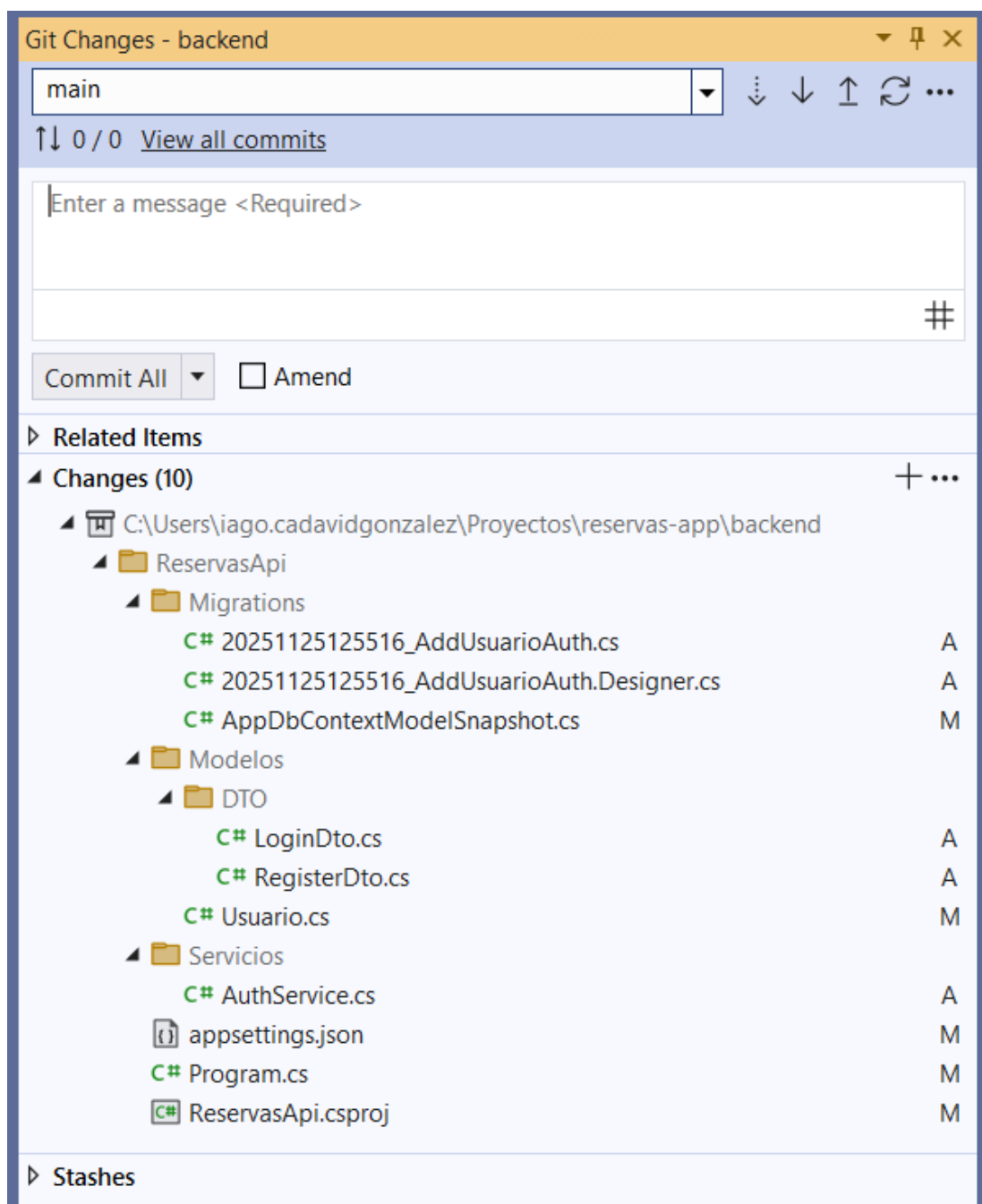
1. Create a "User" model poniendo pwhash y pwsalt en lugar de raw string pw (esto pasa para DTORegistro y DTOlogin).¶
2. Add DbSet<Usuario> in your AppDbContext (ya lo tenía). **Run migrations to update db.**¶
3. Create DTOs for Register/Login¶
4. Create a service for pw hashing + JWT token generation i.e. create a folder Services with file AuthService.cs.¶

..... Page Break

5. **Add JWT configuration in appsettings.json** (del backend)¶
6. **Enable JWT Authentication in Program.cs** (del backend)¶
7. Create an **AuthenticationController**¶
8. **Protect your existing CRUD Controllers**¶
9. **Test in Swagger**¶

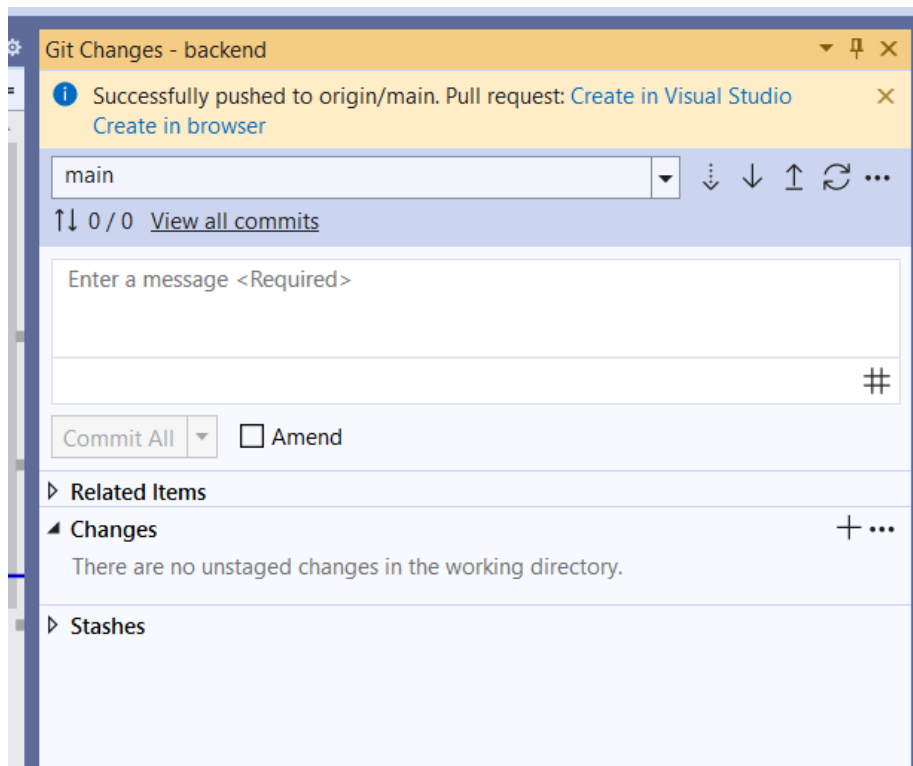
Cambios hasta ahora que voy comitear y pushear: NEXT PAGE

Cambios hasta ahora que voy comitear y pushear:



Puse como mensaje pal commit all “implementación incompleta de autenticación básica.”

NEXT PAGE



Next page

Correo: lago Xesus Cadavid Gonz... (sin asunto) - a22lagocg@iessan... Implementación de autenticación... Plextime Commits · lagoPol/reservas-backe...

https://github.com/lagoPol/reservas-backend/commits/main/

lagoPol / reservas-backend

Search Type to search

Code Issues Pull requests Actions Projects Wiki Security Insights Settings

Commits

main All users All time

Commits on Nov 25, 2025

Implementación incompleta de autenticación básica.
lagoPol committed 2 minutes ago769ace4

Commits on Nov 21, 2025

Comento Kafka, hago xUnit test get /api /reservas
lagoPol committed 4 days ago30b4d06

Commits on Nov 17, 2025

Añado el código kafka a mi backend.
lagoPol committed last weekdbcc474

Commits on Nov 13, 2025

2nd update a semana 2
lagoPol committed 2 weeks ago4a0ecf4

Commits on Nov 10, 2025

Buscar

15:57 25/11/2025